HTTP authentication

HTTP provides a general framework for access control and authentication. This page is an introduction to the HTTP framework for authentication, and shows how to restrict access to your server using the HTTP "Basic" schema.

The general HTTP authentication framework

RFC 7235 defines the HTTP authentication framework, which can be used by a server to challenge a client request, and by a client to provide authentication information.

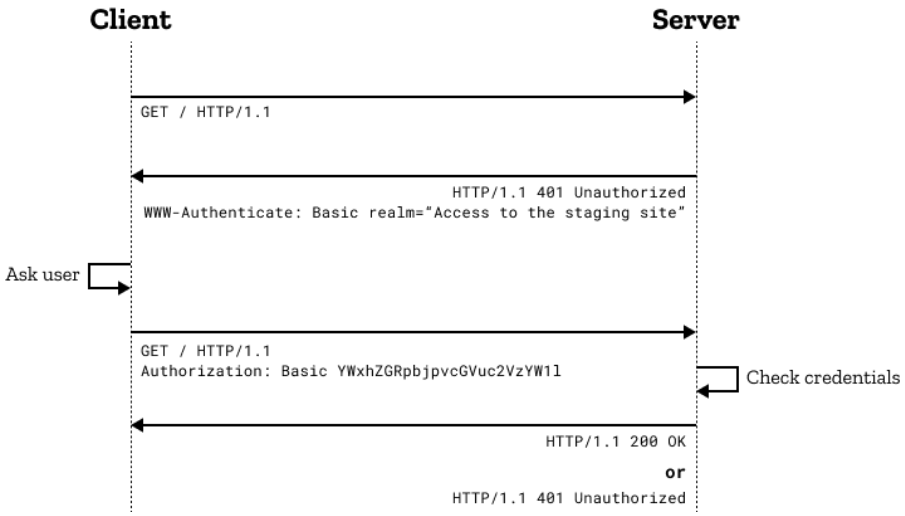
Note: in security protocols, a challenge is some data sent to the client by the server in order to generate a different response each time. Challenge-response protocols are one way to fight against replay attacks where an attacker listens to the previous messages and resends them at a later time to get the same credentials as the original message.

The challenge and response flow works like this:

1. The server responds to a client with a `401` (Unauthorized) response status and provides information on how to authorize with a `WWW-Authenticate` response header containing at least one challenge.
2. A client that wants to authenticate itself with the server can then do so by including an `Authorization` request header with the credentials.
3. Usually a client will present a password prompt to the user and will then issue the request including the correct `Authorization` header.

The general message flow below is the same for most (if not all) authentication schemes. The actual information in the headers and the way it is encoded does change!

Warning: The "Basic" authentication scheme used in the diagram below sends the credentials encoded but not encrypted. This would be completely insecure unless the exchange was over a secure connection (HTTPS/TLS).



Proxy authentication

The same challenge and response mechanism can be used for proxy authentication. As both resource authentication and proxy authentication can coexist, a different set of headers and status codes is needed. In the case of proxies, the challenging status code is `407` (Proxy Authentication Required), the `Proxy-Authenticate` response header contains at least one challenge applicable to the proxy, and the `Proxy-Authorization` request header is used for providing the credentials to the proxy server.

Access forbidden

If a (proxy) server receives invalid credentials, it should respond with a `401 Unauthorized` or with a `407 Proxy Authentication Required`, and the user may send a new request or replace the Authorization header field.

If a (proxy) server receives valid credentials that are inadequate to access a given resource, the server should respond with the `403 Forbidden` status code. Unlike `401 Unauthorized` or `407 Proxy Authentication Required`, authentication is impossible for this user, and browsers will not propose a new attempt.

In all cases, the server may prefer returning `a 404 Not Found` status code, to hide the existence of the page to a user without adequate privileges or not correctly authenticated.

Authentication of cross-origin images

A potential security hole (that has since been fixed in browsers) was the authentication of cross-site images. From Firefox 59 onwards, image resources loaded from different origins to the current document are no longer able to trigger HTTP authentication dialogs (Firefox bug 1423146), preventing user credentials from being stolen if attackers were able to embed an arbitrary image into a third-party page.

Character encoding of HTTP authentication

Browsers use `utf-8` encoding for usernames and passwords.

Firefox once used `ISO-8859-1`, but changed to `utf-8` for parity with other browsers and to avoid potential problems.

WWW-Authenticate and Proxy-Authenticate headers

The `WWW-Authenticate` and `Proxy-Authenticate` response headers define the authentication method that should be used to gain access to a resource. They must specify which authentication scheme is used so that the client that wishes to authorize knows how to provide the credentials.

The syntax for these headers is the following:

```
WWW-Authenticate: <type> realm=<realm>
Proxy-Authenticate: <type> realm=<realm>
```

Here, `<type>` is the authentication scheme ("Basic" is the most common scheme and introduced below). The realm is used to describe the protected area or to indicate the scope of protection. This could be a message like "Access to the staging site" or similar, so that the user knows to which space they are trying to get access to.

Authorization and Proxy-Authorization headers

The `Authorization` and `Proxy-Authorization` request headers contain the credentials to authenticate a user agent with a (proxy) server. Here, the `<type>` is needed again followed by the

credentials, which can be encoded or encrypted depending on which authentication scheme is used.

```
Authorization: <type> <credentials>
Proxy-Authorization: <type> <credentials>
```

Authentication schemes

The general HTTP authentication framework is the base for a number of authentication schemes.

IANA maintains a list of authentication schemes, but there are other schemes offered by host services, such as Amazon AWS.

Some common authentication schemes include:

Basic
See RFC 7617, base64-encoded credentials. More information below.

Bearer
See RFC 6750, bearer tokens to access OAuth 2.0-protected resources

Digest
See RFC 7616. Firefox 93 and later support the SHA-256 algorithm. Previous versions only support MD5 hashing (not recommended).

HOBA
See RFC 7486, Section 3, **H**TTP **O**rigin-**B**ound **A**uthentication, digital-signature-based

Mutual
See RFC 8120

Negotiate / **NTLM**
See RFC4599

VAPID
See RFC 8292

SCRAM
See RFC 7804

AWS4-HMAC-SHA256
See AWS docs. This scheme is used for AWS3 server authentication.

Schemes can differ in security strength and in their availability in client or server software.

The "Basic" authentication scheme offers very poor security, but is widely supported and easy to set up. It is introduced in more detail below.

Basic authentication scheme

The "Basic" HTTP authentication scheme is defined in RFC 7617, which transmits credentials as user ID/password pairs, encoded using base64.

Security of basic authentication

As the user ID and password are passed over the network as clear text (it is base64 encoded, but base64 is a reversible encoding), the basic authentication scheme **is not secure**. HTTPS/TLS should be used with basic authentication. Without these additional security enhancements, basic authentication should not be used to protect sensitive or valuable information.

Restricting access with Apache and basic authentication

To password-protect a directory on an Apache server, you will need a `.htaccess` and a `.htpasswd` file.

The `.htaccess` file typically looks like this:

```
AuthType Basic
AuthName "Access to the staging site"
AuthUserFile /path/to/.htpasswd
Require valid-user
```

The `.htaccess` file references a `.htpasswd` file in which each line consists of a username and a password separated by a colon (:). You cannot see the actual passwords as they are hashed (using MD5-based hashing, in this case). Note that you can name your `.htpasswd` file

differently if you like, but keep in mind this file shouldn't be accessible to anyone. (Apache is usually configured to prevent access to `.ht*` files).

```
aladdin:$apr1$ZjTqBB3f$IF9gdYAGlMrs2fuINjHsz.
user2:$apr1$O04r.y2H$/vEkesPhVInBByJUkXitA/
```

Restricting access with Nginx and basic authentication

For Nginx, you will need to specify a location that you are going to protect and the `auth_basic` directive that provides the name to the password-protected area. The `auth_basic_user_file` directive then points to a `.htpasswd` file containing the encrypted user credentials, just like in the Apache example above.

```
location /status {
    auth_basic           "Access to the staging site";
    auth_basic_user_file /etc/apache2/.htpasswd;
}
```

Access using credentials in the URL

Many clients also let you avoid the login prompt by using an encoded URL containing the username and the password like this:

```
https://username:password@www.example.com/
```

The use of these URLs is deprecated. In Chrome, the `username:password@` part in URLs is even stripped out for security reasons. In Firefox, it is checked if the site actually requires authentication and if not, Firefox will warn the user with a prompt "You are about to log in to the site `www.example.com` with the username `username`, but the website does not require authentication. This may be an attempt to trick you."

Note: Authorization is not the same as authentication. Authentication is like proving your identity to a website - like entering your username and password to log in. Authorization: Once you're logged in, authorization decides what parts of the website you're allowed to access or what actions you can perform - like being given permission.

This lesson was adapted from MDN Web Docs: https://developer.mozilla.org/en-US/docs/Web/HTTP/Authentication